Università degli Studi di Firenze

Dipartimento di Ingegneria dell'Informazione

Applicativo Java EscapeManager

Autore:
Adriano Luca Castaldo

N° Matricola:
7138545

Corso principale:
Ingegneria del software

Docente corso:
Enrico Vicario

Anno Accademico 2025/2026

Indice

1 Introduzione **1**
1.1 Statement . . . 1
1.2 Stack Tecnologico e Strumenti . . . 1
1.3 Note metodologiche e Uso dell'Intelligenza Artificiale . . . 2
1.4 Struttura del documento . . . 2
1.5 Limiti del prototipo e sviluppi futuri . . . 2

2 Analisi dei Requisiti **3**
2.1 Use Case Diagram (High-Level) . . . 3
2.2 Use Case Diagram (Dettagliato) . . . 4
2.3 Use Case Templates . . . 4
2.3.1 UC1 - Prenotazione Stanza . . . 5
2.3.2 UC2 - Iscrizione Lista d'Attesa . . . 6
2.3.3 UC3 - Aggiornamento Stato Stanza . . . 7
2.3.4 UC4 - Configurazione Tariffe Stanza . . . 8
2.4 Mockup e Interfaccia Utente (UI) . . . 8

3 Progettazione Architetturale e di Dominio **11**
3.1 Domain Model (Class Diagram) . . . 11
3.1.1 Scelte Architetturali e Design Pattern . . . 11
3.2 Progettazione del Database . . . 12
3.2.1 Diagramma Entity-Relationship . . . 12
3.2.2 Strategie di Mapping e Modello Logico . . . 13
3.3 Dinamica del Sistema (Implementation Perspective) . . . 14
3.4 Pattern Creazionali: DAO Factory . . . 15

4 Implementazione e Architettura del Codice **16**
4.1 Architettura dei Package . . . 16
4.2 Livello di Persistenza (Pattern DAO) . . . 16
4.3 Implementazione dei Design Pattern . . . 17
4.3.1 State Pattern . . . 17
4.3.2 Strategy Pattern . . . 18
4.3.3 Observer Pattern . . . 18
4.4 Gestione Centralizzata delle Eccezioni (Information Hiding) . . . 18

5 Collaudo e Testing **20**
5.1 Testing Strutturale (White-Box) . . . 20
5.2 Testing di Integrazione (DAO e Database) . . . 20
5.3 Testing Funzionale (Black-Box e Tracciabilità) . . . 21
5.4 Conclusioni . . . 22

1. Introduzione

1.1 Statement

Il progetto **EscapeManager** modella un sistema software centralizzato per la gestione operativa e commerciale di una catena in *franchising* di Escape Room. L'obiettivo principale dell'applicativo è fornire un'infrastruttura unificata che permetta ai clienti di esplorare le stanze tematiche disponibili ed effettuare prenotazioni e, allo stesso tempo, metta a disposizione del personale strumenti avanzati per il controllo delle sessioni di gioco e la gestione commerciale.

Le sedi della catena ospitano diverse stanze fisiche (Room), ciascuna caratterizzata da un tema specifico, un livello di difficoltà, una capienza massima di giocatori e un ciclo di vita operativo ben definito (es. *Disponibile, In Corso, In Manutenzione, In Pulizia*).

Il sistema è progettato per supportare tre tipologie principali di utenti:

- **Cliente:** può visualizzare il catalogo globale delle stanze filtrando per sede o difficoltà, prenotare una sessione di gioco per la propria squadra in uno slot temporale libero, e iscriversi a un sistema di avvisi per ricevere notifiche in tempo reale qualora si liberasse un orario precedentemente al completo.

- **Game Master (Operatore):** assegnato a una specifica filiale, gestisce il flusso operativo quotidiano. Può monitorare le prenotazioni del giorno, avviare o terminare le sessioni di gioco, aggiornare dinamicamente lo stato delle stanze (es. bloccando una stanza per un guasto tecnico a un enigma) e registrare l'esito della partita.

- **Admin (Amministratore):** possiede una visione d'insieme sull'intera catena. Ha i privilegi per aggiungere nuove filiali, inserire nuovi temi per le stanze e configurare politiche di prezzo dinamiche (es. tariffe base o maggiorate).

Per garantire manutenibilità e scalabilità, l'architettura segue il pattern BCE (Boundary-Control-Entity, approfondito nel Capitolo 3) e fa uso di principi di Object-Oriented Design e di Design Pattern strutturali e comportamentali per risolvere problemi specifici del dominio, come la gestione delle variazioni di stato delle stanze, l'applicazione delle tariffe e il sistema di notifiche. La persistenza dei dati, fondamentale per evitare sovrapposizioni nelle prenotazioni e mantenere lo storico delle partite, è gestita tramite un database relazionale connesso al sistema.

1.2 Stack Tecnologico e Strumenti

Per la realizzazione del sistema sono state adottate tecnologie standard in ambito enterprise, al fine di garantire robustezza e manutenibilità:

- **Linguaggio di Programmazione:** Java (JDK 17+), scelto per il forte supporto al paradigma Object-Oriented e l'ampia disponibilità di librerie.

- **Persistenza (RDBMS):** PostgreSQL. L'interazione con il database avviene tramite connettore JDBC, con un layer DAO (Data Access Object) che si occupa del mapping manuale delle entità di dominio sulle tabelle relazionali.

- **Build Automation:** Maven, utilizzato per la gestione del ciclo di vita del software e delle dipendenze.

- **Testing Framework:** JUnit 5 (Jupiter), impiegato per la stesura di test unitari (strutturali) e test di integrazione (funzionali).
- **Versioning e IDE:** Git con hosting repository su GitHub. L'ambiente di sviluppo integrato utilizzato è Visual Studio Code (VS Code).
- **Modellazione UML:** PlantUML, integrato direttamente nell'IDE per generare diagrammi come codice (*diagrams-as-code*), facilitando il tracciamento delle modifiche nel versioning.

1.3 Note metodologiche e Uso dell'Intelligenza Artificiale

In linea con le moderne pratiche di Ingegneria del Software e con le direttive del corso, lo sviluppo di *EscapeManager* è stato coadiuvato dall'uso consapevole di strumenti di Intelligenza Artificiale Generativa (GitHub Copilot e Google Gemini).

L'approccio adottato non è stato di delega totale, ma di *AI-Assisted Pair Programming*:

- **Architettura e Design:** Le scelte architetturali (pattern BCE, identificazione dei Design Pattern GoF come State, Strategy e Observer) e la modellazione concettuale sono state guidate da ragionamento umano basato sui principi del corso.
- **Generazione del Codice:** L'IA è stata impiegata come "agente" per la stesura del codice boilerplate (es. implementazione dei DAO, getter/setter, refactoring di massa), per l'impalcatura dei test JUnit e per l'isolamento del database nei test di integrazione, sempre previa fornitura di *prompt* ingegnerizzati e rigorosa code review umana.
- **Documentazione:** L'IA è stata utilizzata come supporto per la formattazione tipografica in LaTeX e per la revisione lessicale del presente documento.

1.4 Struttura del documento

Il presente documento è organizzato come segue: il Capitolo 2 presenta l'analisi dei requisiti funzionali e non funzionali; il Capitolo 3 descrive la progettazione architetturale e di dominio; il Capitolo 4 illustra i dettagli implementativi e l'architettura del codice; infine, il Capitolo 5 è dedicato al collaudo e al testing del sistema.

1.5 Limiti del prototipo e sviluppi futuri

Allo stato attuale, *EscapeManager* è un *proof of concept* pienamente funzionale a livello di Business Logic e persistenza dati. L'interazione utente è simulata per semplicità attraverso una Command Line Interface (CLI) o casi di test diretti. Sviluppi futuri prevedono l'esposizione di API REST per un frontend Web/Mobile o, in alternativa, l'implementazione di un'interfaccia grafica desktop tramite JavaFX, mantenendo intatto l'attuale Domain Model grazie al rigoroso disaccoppiamento garantito dall'architettura BCE.

2. Analisi dei Requisiti

Per definire i requisiti funzionali del sistema in ottica Object-Oriented, si è proceduto all'identificazione degli attori principali e dei relativi casi d'uso. La modellazione è stata divisa in due livelli di astrazione: un diagramma di contesto ad alto livello per identificare i macro-obiettivi (User Goals) e un diagramma dettagliato per l'analisi tecnica.

Il sistema prevede le seguenti figure:

- **Cliente:** Utente finale che interagisce col sistema per consultare il catalogo e prenotare le esperienze.
- **Game Master:** Personale di filiale deputato al controllo operativo delle stanze e delle sessioni.
- **Admin:** Amministratore di sistema incaricato di definire il catalogo e le logiche commerciali.

2.1 Use Case Diagram (High-Level)

La Figura 2.1 illustra il diagramma di contesto. Questo livello omette le dipendenze strutturali per evidenziare esclusivamente il macro-scopo di ciascun attore all'interno dell'ecosistema EscapeManager.

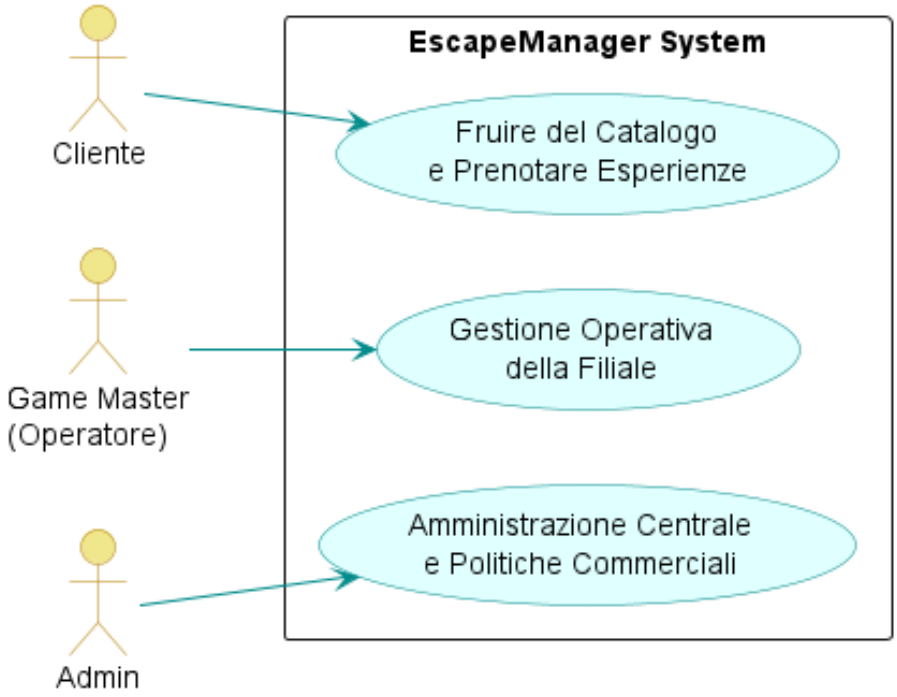

Figura 2.1: Diagramma di Contesto (Macro-obiettivi degli attori)

2.2 Use Case Diagram (Dettagliato)

La Figura 2.2 espande i macro-obiettivi nei casi d'uso effettivi del sistema. Per mantenere il diagramma ad un alto livello di coesione ed evitare l'anti-pattern della decomposizione funzionale, le operazioni di CRUD standard e l'autenticazione di base sono state omesse o raggruppate. In questa vista viene invece evidenziata la relazione di estensione («`extend`») che modella un flusso alternativo cruciale per il dominio aziendale: la gestione opzionale delle liste d'attesa in caso di esaurimento degli slot disponibili.

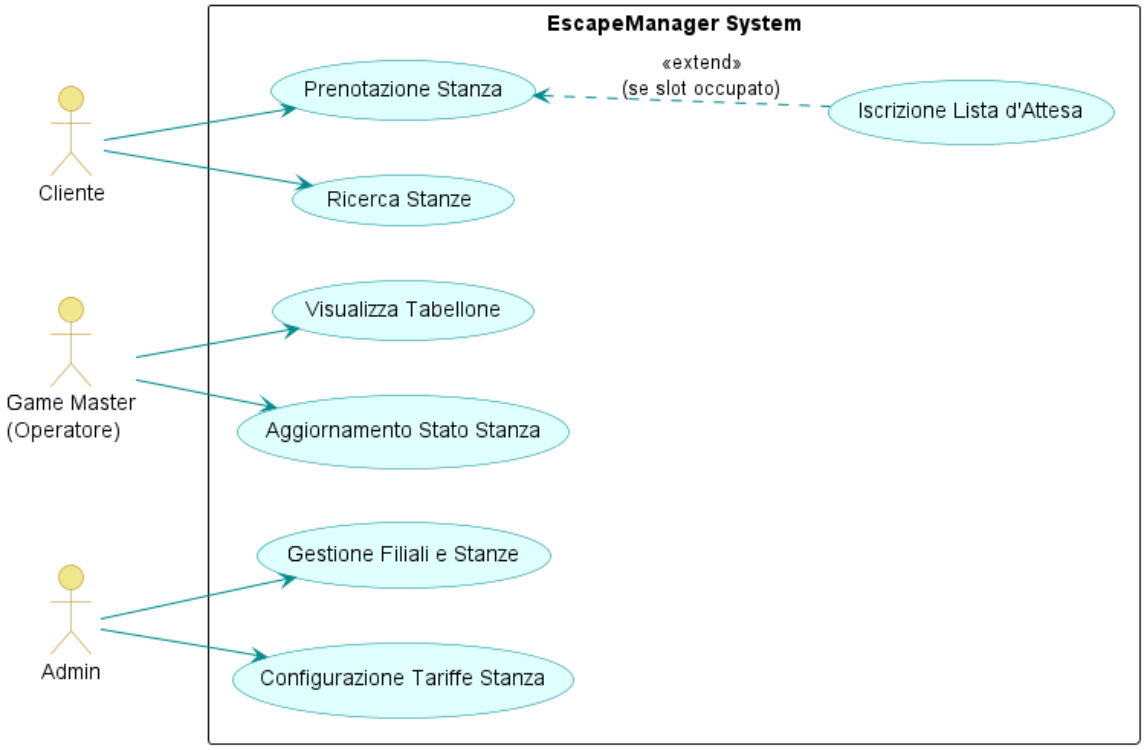

Figura 2.2: Diagramma dei Casi d'Uso Dettagliato

2.3 Use Case Templates

Di seguito vengono dettagliati i template per i casi d'uso primari del sistema, adottando un formato strutturato che esplicita le pre-condizioni, le post-condizioni, il flusso base (Main Success Scenario) e le possibili deviazioni (Alternative Flows). Questa formalizzazione costituisce il contratto per lo sviluppo e la base per la successiva stesura dei test funzionali.

2.3.1 UC1 - Prenotazione Stanza

Nome Use Case	UC1 - Prenotazione Stanza
Attore Principale	Cliente
Livello	User Goal
Scopo	Permettere al cliente di riservare una stanza in uno specifico slot temporale, calcolando la tariffa appropriata.
Pre-condizioni	Il Cliente è autenticato nel sistema. Il sistema ha caricato il catalogo delle stanze.
Post-condizioni	Una nuova Prenotazione viene registrata nel sistema con stato "Confermata". Lo slot orario della stanza risulta occupato.

Flusso Base (Basic Flow)

1. Il Cliente seleziona una Stanza dal catalogo e richiede di visualizzare le disponibilità.
2. Il Sistema mostra il calendario con le date e gli slot orari disponibili per la Stanza scelta.
3. Il Cliente seleziona una data, uno slot orario libero e inserisce il numero di giocatori.
4. Il Sistema calcola il prezzo totale basandosi sulla strategia di prezzo corrente (es. base, weekend) e mostra il riepilogo.
5. Il Cliente conferma la prenotazione.
6. Il Sistema registra la prenotazione, aggiorna lo stato dello slot orario e mostra un messaggio di successo.

Flussi Alternativi (Alternative Flows)

3a. Slot orario non più disponibile (Race condition):

3a.1 Se nel momento della conferma lo slot è stato appena prenotato da un altro utente, il Sistema notifica l'errore.

3a.2 Il Sistema propone al Cliente di tornare al passo 2 o di iscriversi alla Lista d'Attesa (UC2 - *Extend*).

4a. Numero di giocatori non valido:

4a.1 Se il numero di giocatori inserito supera la capienza massima (o è inferiore alla minima) della Stanza, il Sistema segnala l'errore.

4a.2 Il Cliente è riportato al passo 3 per correggere il dato.

Tabella 2.1: Template del Caso d'Uso: Prenotazione Stanza

2.3.2 UC2 - Iscrizione Lista d'Attesa

Nome Use Case	UC2 - Iscrizione Lista d'Attesa
Attore Principale	Cliente
Livello	User Goal
Scopo	Permettere a un cliente di iscriversi alla lista d'attesa per una stanza attualmente non disponibile, per essere notificato in caso di liberazione.
Pre-condizioni	Il Cliente è autenticato. La Stanza selezionata non ha slot disponibili per la data richiesta.
Post-condizioni	Il Cliente è registrato come Observer nella WaitingList della Stanza.

Flusso Base (Basic Flow)

1. Il Cliente visualizza che la Stanza desiderata è completamente prenotata.
2. Il Cliente seleziona l'opzione "Iscriviti alla Lista d'Attesa".
3. Il Sistema richiede la conferma per l'attivazione delle notifiche.
4. Il Cliente conferma.
5. Il Sistema aggiunge il Cliente agli observer della stanza e mostra un messaggio di successo.

Flussi Alternativi (Alternative Flows)

4a. Il Cliente annulla l'iscrizione:

4a.1 Il Cliente seleziona "Annulla" alla richiesta di conferma.

4a.2 Il Sistema interrompe l'operazione e riporta il Cliente al catalogo.

5a. Cliente già in lista:

5a.1 Il Sistema rileva che il Cliente è già registrato come observer per quella Stanza.

5a.2 Il Sistema notifica l'utente dell'iscrizione già attiva, senza duplicare la registrazione.

2.3.3 UC3 - Aggiornamento Stato Stanza

Nome Use Case	UC3 - Aggiornamento Stato Stanza
Attore Principale	Game Master
Livello	User Goal
Scopo	Aggiornare il ciclo di vita operativo di una stanza (es. da Disponibile a In Corso, da In Corso a In Pulizia).
Pre-condizioni	Il Game Master è autenticato e autorizzato per la sede di competenza.
Post-condizioni	Lo stato della Stanza è aggiornato nel sistema persistente.

Flusso Base (Basic Flow)

1. Il Game Master accede alla dashboard di controllo delle stanze.
2. Il Sistema mostra l'elenco delle stanze della sede e il loro stato attuale.
3. Il Game Master seleziona una Stanza in stato "Disponibile" e clicca su "Avvia Sessione".
4. Il Sistema valida la transizione di stato.
5. Il Sistema aggiorna lo stato della stanza in "In Corso" e registra l'evento.

Flussi Alternativi (Alternative Flows)

4a. Transizione di stato non valida:

4a.1 Il Sistema rileva che l'azione richiesta viola il pattern di stato (es. tentativo di avviare una stanza "In Manutenzione").

4a.2 Il Sistema lancia un'eccezione interna e mostra al Game Master un messaggio di errore: "Transizione non consentita dallo stato attuale".

2.3.4 UC4 - Configurazione Tariffe Stanza

Nome Use Case	UC4 - Configurazione Tariffe Stanza
Attore Principale	Admin
Livello	User Goal
Scopo	Assegnare o modificare la strategia di prezzo applicata a una specifica stanza.
Pre-condizioni	L'Admin è autenticato. La Stanza esiste nel sistema.
Post-condizioni	La nuova strategia di prezzo (PricingStrategy) è associata alla Stanza.
Flusso Base (Basic Flow)	
1. L'Admin accede al pannello di amministrazione e seleziona una Stanza. 2. Il Sistema mostra i dettagli della stanza e l'attuale strategia di prezzo. 3. L'Admin seleziona l'opzione per modificare la tariffa e sceglie una nuova strategia dall'elenco (es. *Tariffa Base*, *Tariffa Dinamica*). 4. L'Admin salva la configurazione. 5. Il Sistema aggiorna i dati nel database e conferma il successo dell'operazione.	

2.4 Mockup e Interfaccia Utente (UI)

Al fine di concretizzare l'analisi dei requisiti e validare le interazioni previste nei casi d'uso principali, sono stati realizzati dei mockup per le schermate core del sistema. Questa fase di *prototipazione low-fidelity* garantisce una mappatura visiva diretta tra le intenzioni dell'attore (User Goal) e le effettive funzionalità esposte dal sistema *EscapeManager*.

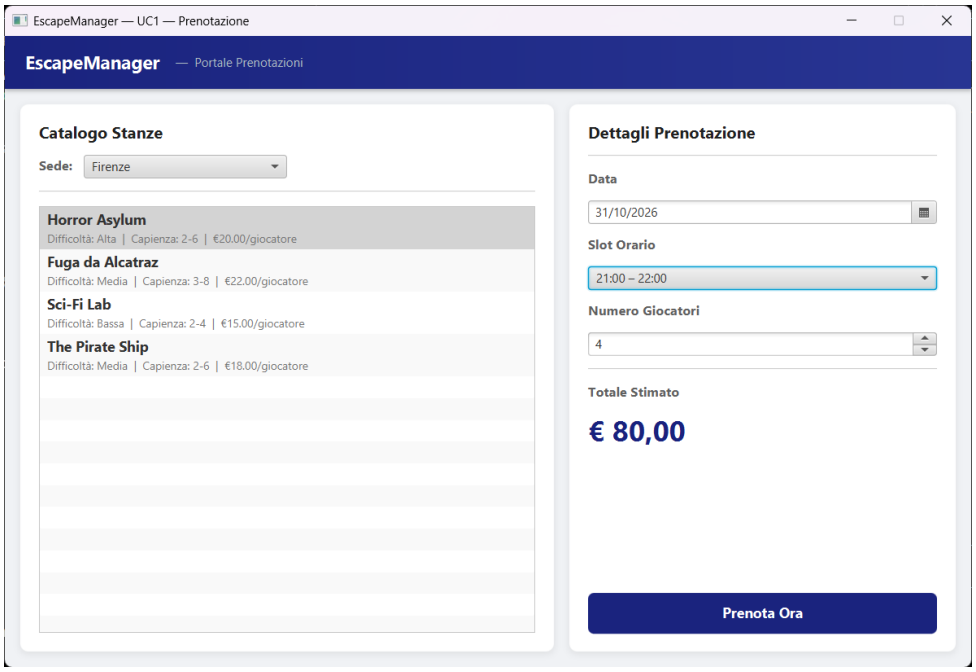

Figura 2.3: Interfaccia Cliente: Visualizzazione disponibilità e preventivo (UC1)

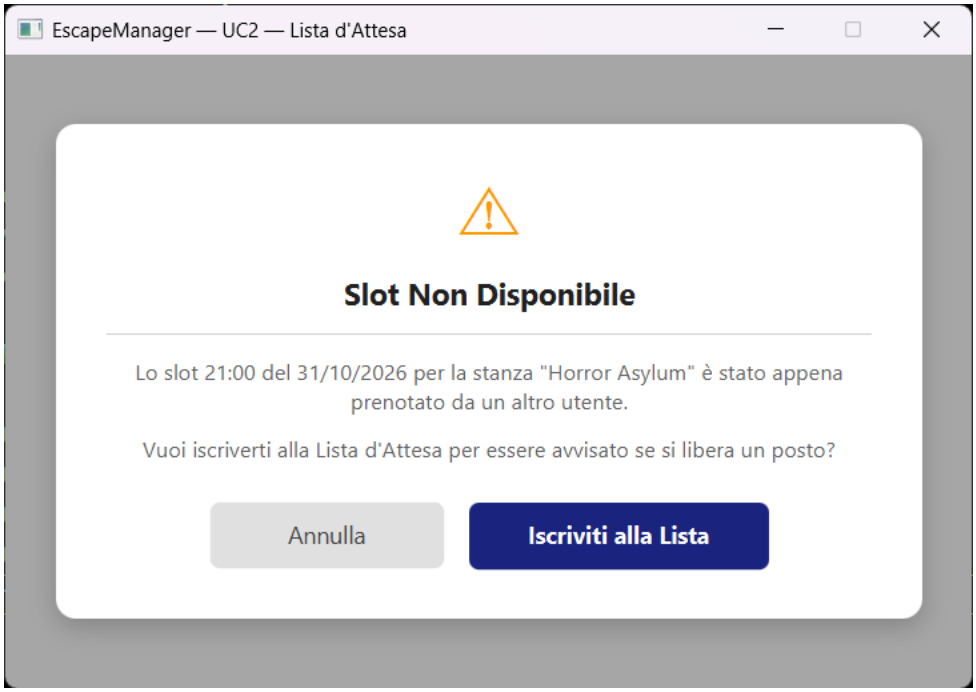

Figura 2.4: Interfaccia Cliente: Prompt di iscrizione alla Lista d'Attesa (UC2)

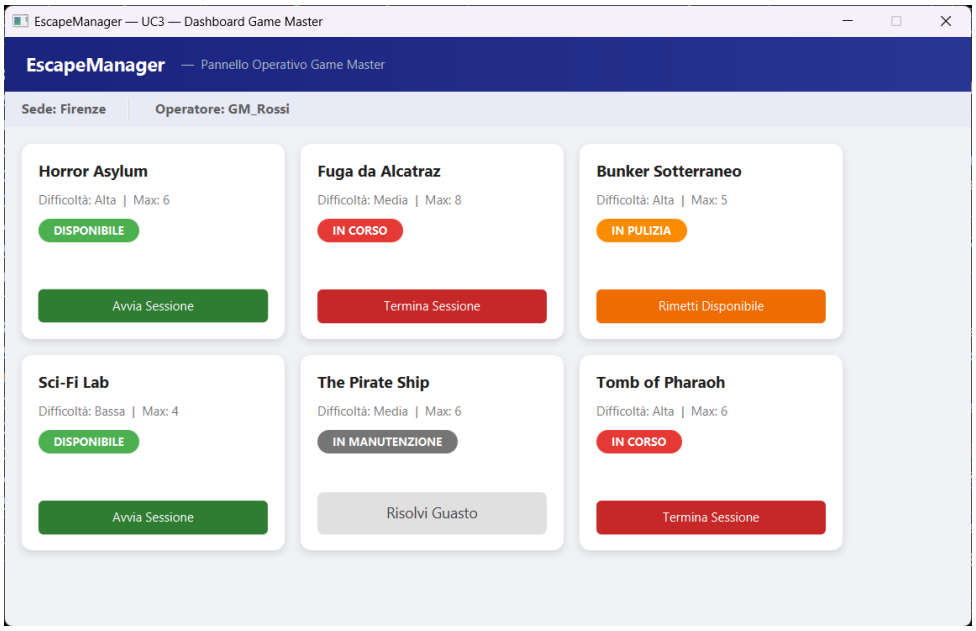

Figura 2.5: Interfaccia Game Master: Tabellone operativo e controllo stati (UC3)

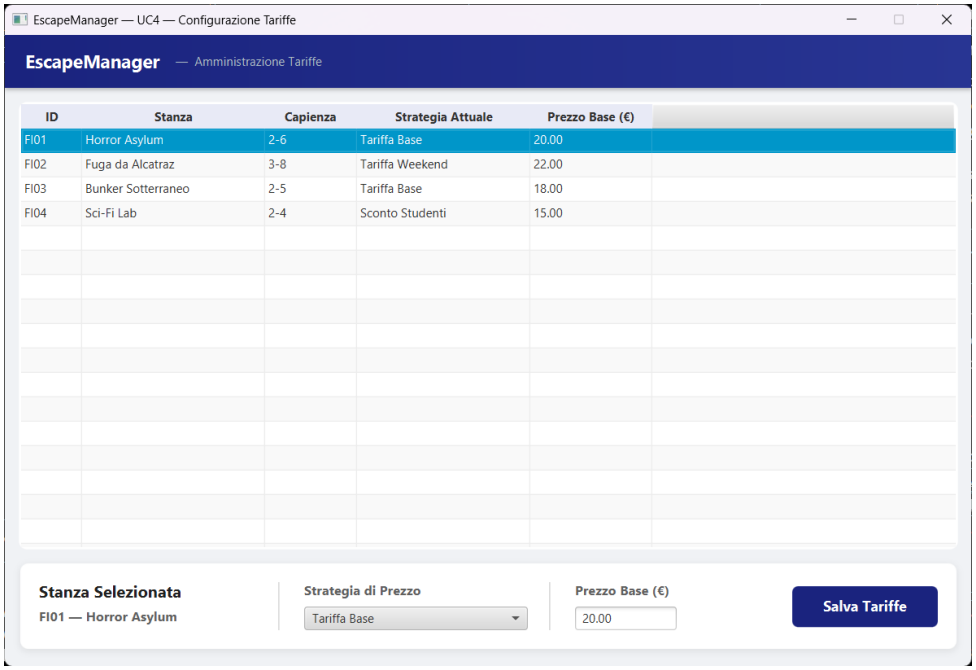

Figura 2.6: Interfaccia Admin: Configurazione delle strategie di prezzo (UC4)

3. Progettazione Architetturale e di Dominio

Il processo di transizione dall'analisi dei requisiti alla progettazione ha seguito i principi dell'Object-Oriented Design (OOD) e l'approccio BCE (Boundary-Control-Entity). In questo capitolo viene dettagliata la struttura statica del Domain Model, ponendo particolare enfasi sull'applicazione dei Design Pattern fondamentali per garantire la manutenibilità, l'estensibilità e il disaccoppiamento del sistema.

3.1 Domain Model (Class Diagram)

La Figura 3.1 illustra il diagramma delle classi relativo al nucleo logico dell'applicativo (Entity layer). Il modello non include volutamente i dettagli legati all'accesso ai dati (DAO) o all'interfaccia utente (View/CLI), concentrandosi esclusivamente sulle regole di business del franchise.

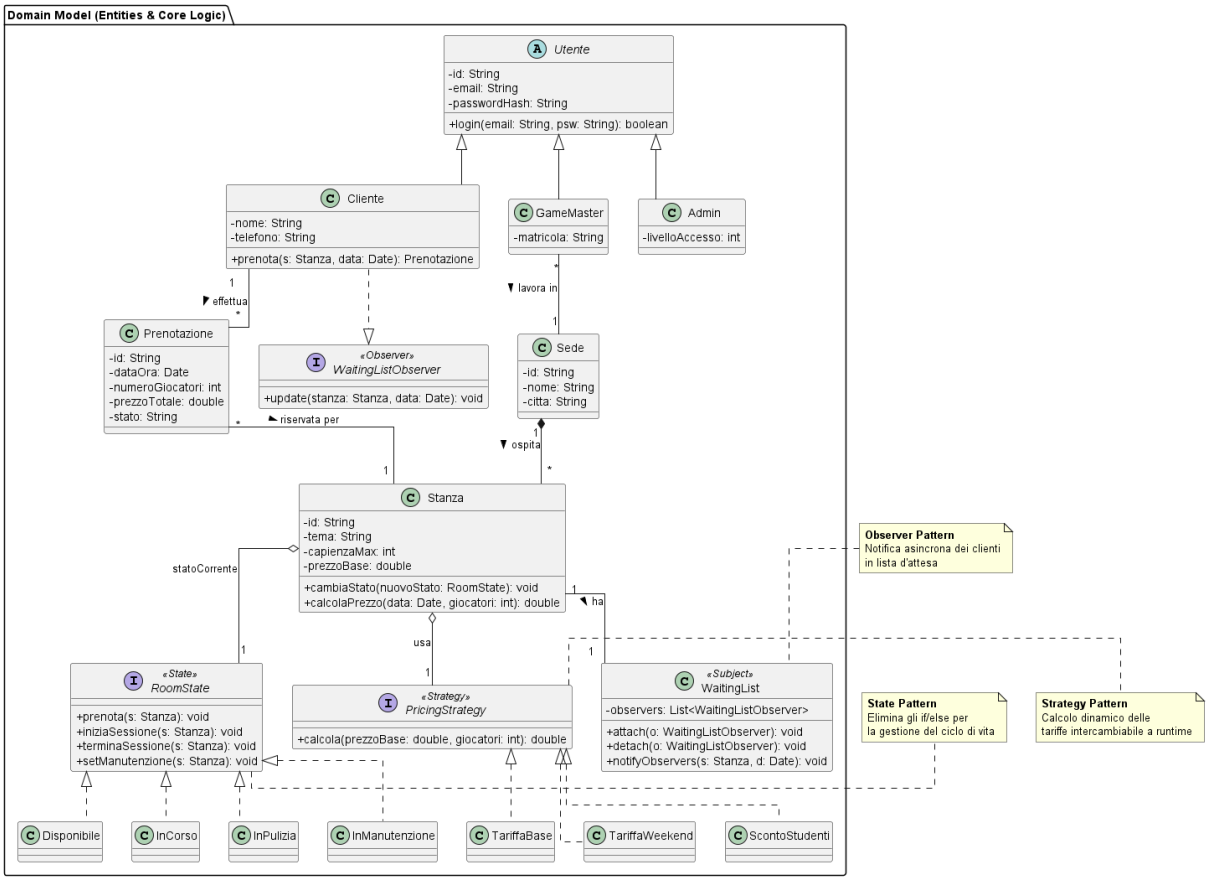

Figura 3.1: Diagramma delle Classi del Domain Model con Design Pattern

3.1.1 Scelte Architetturali e Design Pattern

L'architettura del dominio è stata concepita per risolvere nativamente le complessità emerse durante l'analisi dei casi d'uso (Capitolo 2), ricorrendo a tre Design Pattern della *Gang of Four*

(GoF):

- **State Pattern (Gestione ciclo di vita):** Come emerso nell'UC3, la `Stanza` attraversa fasi operative rigide (Disponibile, In Corso, In Pulizia, In Manutenzione). Invece di delegare il controllo a complessi blocchi `switch/case` basati su stringhe o enumerazioni, lo stato è stato reificato nell'interfaccia `RoomState`. Questo garantisce il rispetto del principio *Open-Closed*: l'aggiunta di un nuovo stato futuro non richiederà la modifica del codice della classe `Stanza`.
- **Strategy Pattern (Politiche di Prezzo):** Le regole di tariffazione dipendono da fattori esterni (giorno della settimana, festività, tipologia di clienti). La classe `Stanza` delega il calcolo del costo totale all'interfaccia `PricingStrategy`. L'amministratore (UC4) può così iniettare a runtime strategie diverse (es. `TariffaWeekend` o `ScontoStudenti`) variando l'algoritmo di calcolo in modo trasparente.
- **Observer Pattern (Lista d'Attesa asincrona):** Per soddisfare il requisito di notifica asincrona dell'UC2 (Iscrizione Lista d'Attesa), è stato implementato un meccanismo di Publish-Subscribe. La classe `WaitingList` agisce da *Subject*, mantenendo una lista di `WaitingListObserver` (interfaccia implementata dal `Cliente`). Quando una prenotazione viene cancellata, il subject invoca il metodo `update()`, disaccoppiando totalmente la logica di notifica dal flusso principale.

Oltre ai pattern, si sottolinea l'uso dell'ereditarietà per la gestione degli attori (`Utente` come classe astratta generalizzata) e l'impiego della composizione forte (`*-`) tra `Sede` e `Stanza`, indicando che il ciclo di vita logico di una stanza è strettamente legato alla filiale fisica che la ospita.

3.2 Progettazione del Database

Per garantire la persistenza dei dati e supportare l'infrastruttura di un sistema centralizzato, il Domain Model Object-Oriented è stato mappato su un database relazionale (PostgreSQL). La transizione dal modello concettuale allo schema logico ha richiesto l'applicazione di specifiche strategie di *Object-Relational Mapping (ORM)*.

3.2.1 Diagramma Entity-Relationship

La Figura 3.2 mostra il diagramma Entità-Relazione (ER) del database, illustrando le tabelle principali, le chiavi primarie (PK) e i vincoli di chiave esterna (FK).

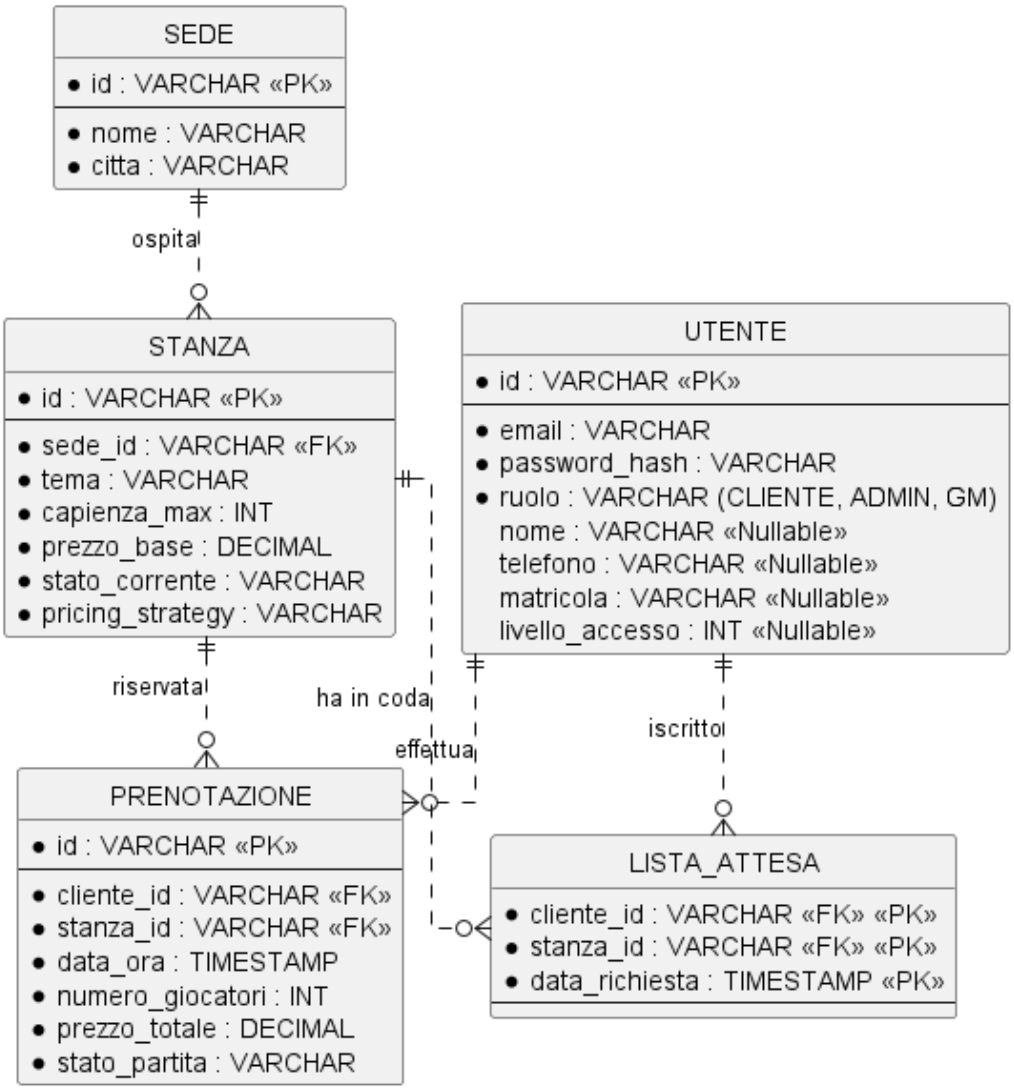

Figura 3.2: Diagramma Entity-Relationship (ER) del sistema

3.2.2 Strategie di Mapping e Modello Logico

Il design relazionale affronta la dicotomia tra il paradigma a oggetti e quello tabellare attraverso le seguenti scelte implementative:

- **Mapping dell'Ereditarietà (Single Table Strategy):** La gerarchia della classe astratta `Utente` (Cliente, GameMaster, Admin) è stata mappata utilizzando il pattern *Single Table Inheritance*. È stata creata un'unica tabella `UTENTE` provvista di una colonna discriminante (`ruolo`). Gli attributi specifici delle sottoclassi (es. `matricola` per il Game Master o `telefono` per il Cliente) sono stati resi *nullable*. Questa scelta ottimizza le query di autenticazione evitando costose operazioni di `JOIN`.

- **Mapping dei Design Pattern (State e Strategy):** Le interfacce `RoomState` e `PricingStrategy` non diventano tabelle separate. All'interno della tabella `STANZA`, esse sono persistite tramite stringhe (es. `stato_corrente = 'IN_MANUTENZIONE'`). A livello di codice Java, i Data Access Object (DAO) utilizzeranno un meccanismo di *Factory* per istanziare l'oggetto concreto corrispondente al valore testuale letto dal database.

- **Associazioni Molti-a-Molti (Observer Pattern):** La logica Publish-Subscribe della Lista d'Attesa (UC2) si traduce relazionalmente nella tabella di *junction* `LISTA_ATTESA`. Questa tabella mappa la relazione N:N tra Clienti e Stanze, utilizzando una chiave primaria composita formata dalle foreign key e dal timestamp della richiesta.

Di seguito viene riportato lo **Schema Logico Relazionale** formalizzato:

- **SEDE**(id, nome, citta)
- **STANZA**(id, sede_id, tema, capienza_max, prezzo_base, stato_corrente, pricing_strategy)
- **UTENTE**(id, email, password_hash, ruolo, nome*, telefono*, matricola*, livello_accesso*) — * *attributo nullable a seconda del ruolo*
- **PRENOTAZIONE**(id, cliente_id, stanza_id, data_ora, numero_giocatori, prezzo_totale, stato_partita)
- **LISTA_ATTESA**(cliente_id, stanza_id, data_richiesta)

3.3 Dinamica del Sistema (Implementation Perspective)

Per illustrare le dinamiche di interazione a *run-time* tra gli oggetti, si fa riferimento all'approccio descritto nello standard UML per la *Implementation Perspective*. La Figura 3.3 mostra il Sequence Diagram relativo al caso d'uso principale (UC1 - Prenotazione Stanza).

Il diagramma evidenzia il rispetto del pattern architetturale BCE: l'interfaccia utente (Boundary) delega l'elaborazione al `PrenotazioneController` (Control), il quale orchestra il reperimento delle risorse persistenti e demanda la logica matematica di calcolo del prezzo all'entità `Stanza` (Entity). Particolare attenzione è stata posta nel disaccoppiamento del Controller dall'accesso ai dati, delegato interamente a un pattern creazionale.

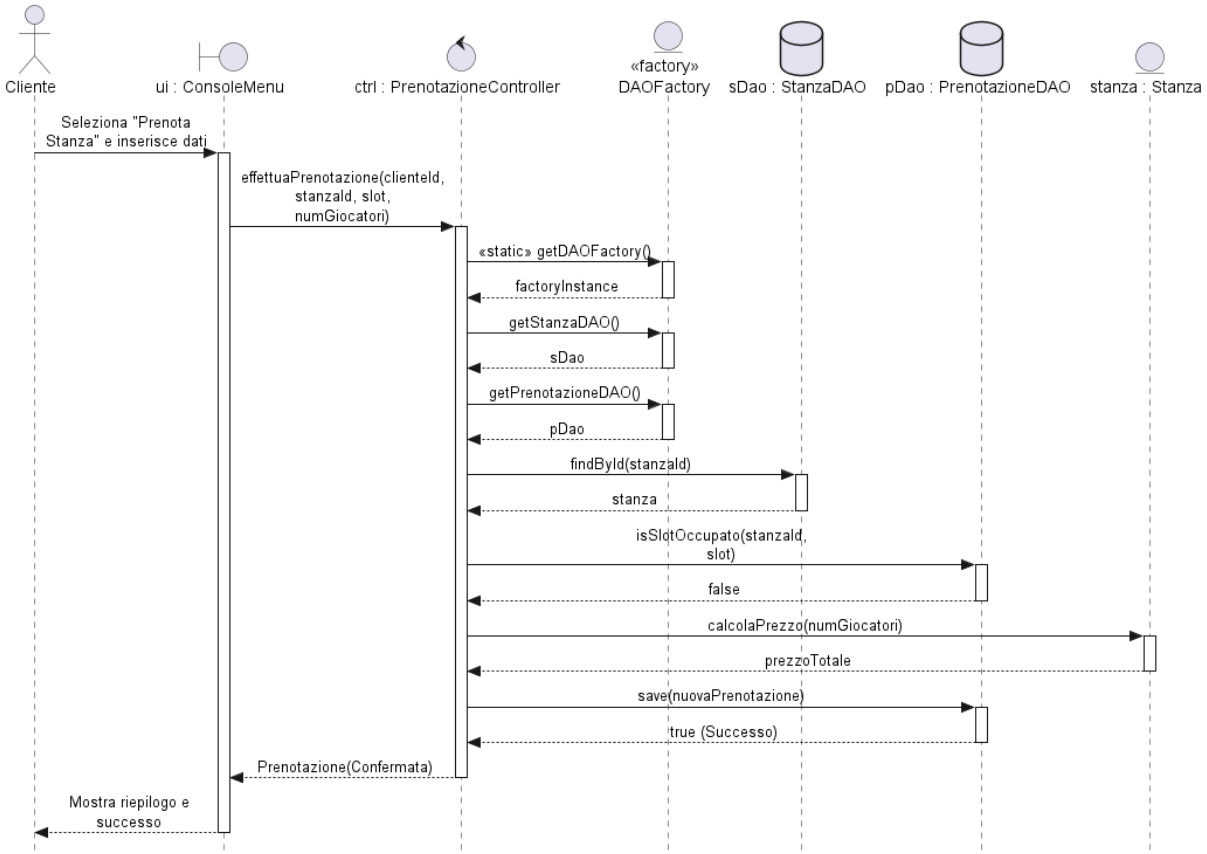

Figura 3.3: Sequence Diagram del caso d'uso UC1 (Prenotazione Stanza)

3.4 Pattern Creazionali: DAO Factory

Come evincibile dal diagramma di sequenza, il sistema adotta il pattern **Abstract Factory** (nella specifica accezione di *DAO Factory*) per la generazione degli oggetti preposti all'accesso ai dati. L'obiettivo di questa scelta progettuale è garantire il totale disaccoppiamento tra la Business Logic (Controller) e l'infrastruttura di persistenza (PostgreSQL). Il Controller non istanzia mai in modo diretto una classe concreta come `StanzaDAOPostgres`, ma si affida all'interfaccia esposta dalla Factory:

```
// Reperimento dinamico dei DAO indipendentemente dal DB sottostante
DAOFactory factory = DAOFactory.getDAOFactory();
StanzaDAO stanzaDao = factory.getStanzaDAO();
PrenotazioneDAO prenotazioneDao = factory.getPrenotazioneDAO();
```

Listing 3.1: Utilizzo del pattern DAO Factory nel Controller

Questa architettura rende il sistema altamente manutenibile ed estensibile: la migrazione verso un diverso DBMS (es. MySQL o un DB NoSQL) richiederebbe unicamente l'implementazione di una nuova `Factory` concreta, lasciando intatto il Domain Model e i Controller.

4. Implementazione e Architettura del Codice

Questo capitolo descrive la traduzione del Domain Model e dello Schema Logico in codice sorgente Java. L'architettura software è stata strutturata seguendo il pattern architetturale *Layered* (a strati), che ricalca logicamente il paradigma BCE (Boundary-Control-Entity), garantendo un elevato disaccoppiamento e facilitando il testing isolato dei componenti.

4.1 Architettura dei Package

L'applicazione è organizzata in quattro package principali, illustrati nel Package Diagram in Figura 4.1. Le dipendenze tra i package sono rigorosamente direzionali: lo strato di presentazione (`cli`) non accede mai direttamente al database, ma passa per i `controllers`, che a loro volta orchestrano il `domain` e si interfacciano con i `dao`.

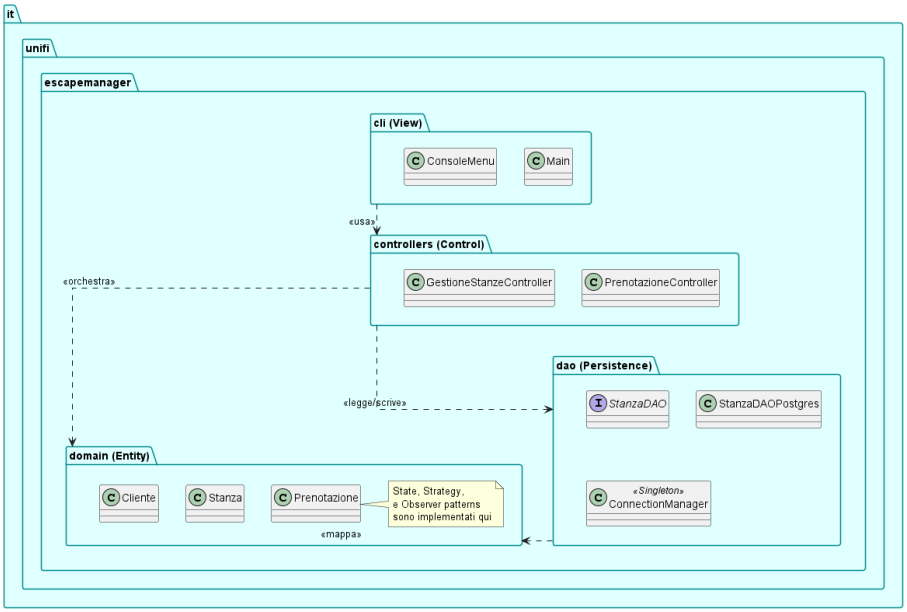

Figura 4.1: Package Diagram dell'architettura di sistema

4.2 Livello di Persistenza (Pattern DAO)

Per soddisfare i requisiti del modulo di Laboratorio di Informatica, l'accesso al database PostgreSQL è stato isolato tramite il pattern **Data Access Object (DAO)**. Questo garantisce che la logica di business rimanga del tutto agnostica rispetto alla tecnologia di persistenza.

La connessione al database è gestita tramite un **Singleton Pattern** (`ConnectionManager`), che assicura l'esistenza di un'unica istanza di connessione (o di un connection pool) condivisa per tutta l'applicazione, evitando l'overhead di connessioni multiple.

```
public interface GenericDAO<T, ID> {
    void save(T entity) throws SQLException;
    T findById(ID id) throws SQLException;
```

```
    List<T> findAll() throws SQLException;
    void update(T entity) throws SQLException;
    void delete(ID id) throws SQLException;
}

public interface StanzaDAO extends GenericDAO<Stanza, String> {
    List<Stanza> findBySedeAndStato(String sedeId, String
        statoCorrente) throws SQLException;
}
```

Listing 4.1: Interfaccia DAO e uso dei Generics

4.3 Implementazione dei Design Pattern

Di seguito vengono riportati i frammenti di codice più significativi che mostrano l'implementazione pratica dei Design Pattern progettati nel Capitolo 3.

4.3.1 State Pattern

Lo State Pattern elimina la necessità di costrutti condizionali complessi per la gestione del ciclo di vita della `Stanza`. Ogni stato concreto implementa il comportamento specifico e gestisce, ove necessario, la transizione allo stato successivo.

```
public interface RoomState {
    void prenota(Stanza stanza);
    void iniziaSessione(Stanza stanza);
    void terminaSessione(Stanza stanza);
    void setManutenzione(Stanza stanza);
    void setDisponibile(Stanza stanza);
    String getStatusString();
}

public class DisponibileState implements RoomState {
    @Override
    public void iniziaSessione(Stanza stanza) {
        stanza.setStato(new InCorsoState()); // Transizione di stato
    }

    @Override
    public void terminaSessione(Stanza stanza) {
        throw new IllegalStateException("Impossibile terminare una
            sessione per una stanza DISPONIBILE.");
    }
    // ... altri metodi
}
```

Listing 4.2: Implementazione dello State Pattern

4.3.2 Strategy Pattern

Il calcolo del prezzo della prenotazione (UC1) è demandato alla `PricingStrategy`. Il Controller, in base alla data della prenotazione, inietta la strategia corretta all'interno dell'entità.

```
public interface PricingStrategy {
    double calcola(double prezzoBase, int numeroGiocatori);
}

public class TariffaWeekend implements PricingStrategy {
    private static final double SOVRAPPREZZO = 1.20; // +20%

    @Override
    public double calcola(double prezzoBase, int numeroGiocatori) {
        return (prezzoBase * numeroGiocatori) * SOVRAPPREZZO;
    }
}
```

Listing 4.3: Implementazione dello Strategy Pattern

4.3.3 Observer Pattern

La lista d'attesa (UC2) implementa un sistema Publisher-Subscriber puro in Java, disaccoppiando l'evento di cancellazione di una prenotazione dall'invio effettivo delle notifiche.

```
public interface WaitingListObserver {
    void update(Stanza stanza, LocalDateTime slotLiberato);
}

public class WaitingList {
    private List<WaitingListObserver> observers = new ArrayList<>();

    public void attach(WaitingListObserver observer) {
        observers.add(observer);
    }

    public void notifyObservers(Stanza stanza, LocalDateTime slot) {
        for (WaitingListObserver obs : observers) {
            obs.update(stanza, slot);
        }
    }
}
```

Listing 4.4: Implementazione dell'Observer Pattern

4.4 Gestione Centralizzata delle Eccezioni (Information Hiding)

Un aspetto cruciale dell'implementazione strutturata ha riguardato la gestione dei flussi alternativi e degli errori. Invece di affidarsi alle generiche eccezioni standard di Java (es.

`RuntimeException`), è stata disegnata una gerarchia di eccezioni custom radicata nella classe `EscapeManagerException`.

Come illustrato nella Figura 4.2, la gerarchia divide formalmente gli errori in base al layer architetturale in cui si verificano:

- **`ValidationException`:** utilizzata dal dominio per segnalare violazioni delle invarianti di business (es. la sottoclasse `SlotNonDisponibileException`).
- **`DatabaseException`:** utilizzata dal layer DAO per incapsulare problemi di persistenza.

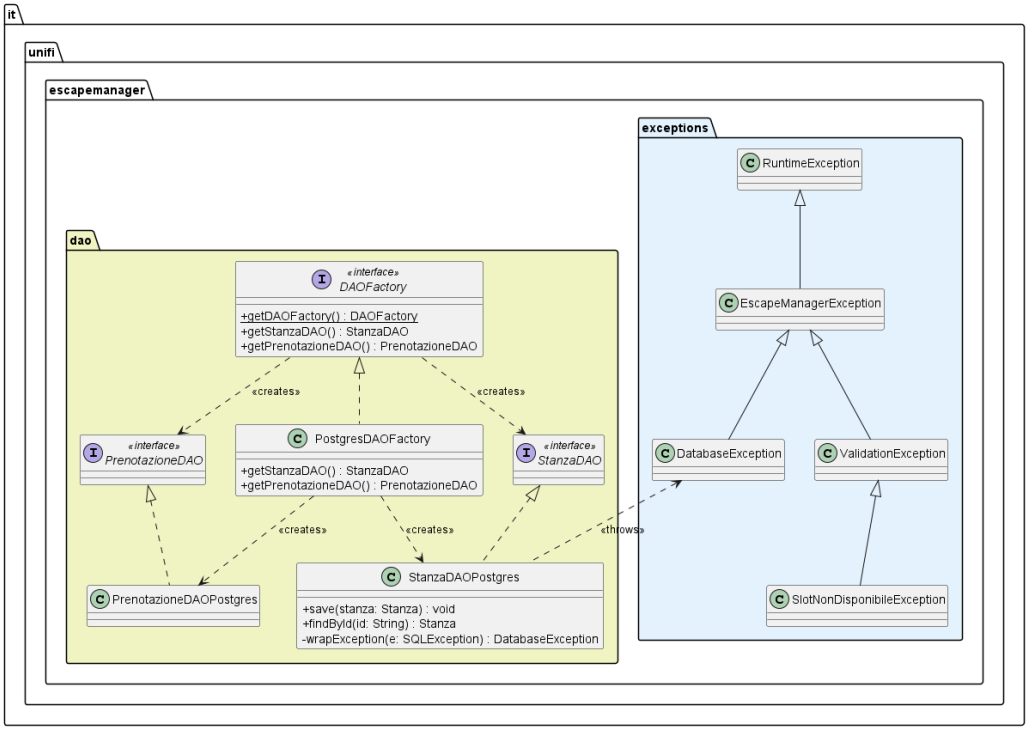

Figura 4.2: Diagramma delle Classi: Pattern DAO Factory e Gerarchia Eccezioni

Questa struttura risponde in modo rigoroso al principio dell'**Information Hiding**. Nel livello DAO, le `SQLException` lanciate dal driver JDBC vengono intercettate, incapsulate in una `DatabaseException` e rilanciate. In questo modo, eventuali stack trace legati a driver SQL o dettagli implementativi del database non inquinano mai i livelli superiori (Boundary e Control).

```
try (PreparedStatement stmt = conn.prepareStatement(query)) {
    // Esecuzione query...
} catch (SQLException e) {
    // La SQLException originaria viene nascosta ai livelli superiori
    throw new DatabaseException("Errore di accesso ai dati durante il
        salvataggio.", e);
}
```

Listing 4.5: Incapsulamento delle eccezioni SQL nel layer DAO

5. Collaudo e Testing

La fase di verifica e validazione del sistema **EscapeManager** è stata condotta seguendo un approccio multilivello, utilizzando il framework **JUnit 5**. L'obiettivo del collaudo non è stato solo quello di garantire l'assenza di difetti evidenti (bug), ma soprattutto di dimostrare la robustezza dell'architettura e la perfetta aderenza ai requisiti funzionali delineati nel Capitolo 2.

5.1 Testing Strutturale (White-Box)

I test strutturali (o Unit Test) si sono concentrati sulle classi del `Domain Model`, con l'obiettivo di massimizzare la copertura (Code Coverage) dei rami condizionali e verificare la correttezza algoritmica, in totale isolamento dalle dipendenze esterne.

Particolare attenzione è stata dedicata al collaudo dei Design Pattern. Il Listing 5.1 mostra un test parametrizzato per la verifica delle politiche di prezzo dinamiche (*Strategy Pattern*).

```
class PricingStrategyTest {

    @Test
    void testTariffaBaseCalcoloCorretto() {
        // Setup
        Stanza stanza = new Stanza("R01", "FI01", "Horror Asylum", 6,
            20.0);
        stanza.setPricingStrategy(new TariffaBase());

        // Execution: 4 giocatori a 20 euro base = 80 euro
        double prezzoCalcolato = stanza.calcolaPrezzo(4);

        // Assertion
        assertEquals(80.0, prezzoCalcolato);
    }
}
```

Listing 5.1: Unit Test per la logica di calcolo del prezzo (Strategy Pattern)

5.2 Testing di Integrazione (DAO e Database)

Per verificare il corretto interfacciamento tra l'applicazione Java e il database PostgreSQL (livello ORM), sono stati implementati dei test di integrazione sulle classi DAO. Per garantire la **ripetibilità** e l'isolamento dei test (evitando che l'esecuzione di un test sporchi i dati del successivo), è stato sfruttato il costrutto `@BeforeEach` di JUnit per lanciare uno script SQL di *reset e re-inizializzazione* del database prima di ogni metodo di test.

```
class StanzaDAOTest {
    private StanzaDAO stanzaDAO;

    @BeforeEach
    void setUpDatabase() throws Exception {
```

```java
        // Esegue il reset e la re-inizializzazione del DB a uno stato noto
        DatabaseHelper.resetTestDatabase();
        stanzaDAO = new StanzaDAOPostgres();
    }

    @Test
    void testFindStanzeDisponibiliBySede() throws SQLException {
        // La sede "FI01" nel db di test ha 1 stanza DISPONIBILE
        List<Stanza> disponibili = stanzaDAO.findBySedeAndStato("FI01", "DISPONIBILE");

        assertNotNull(disponibili);
        assertEquals(1, disponibili.size());
        assertTrue(disponibili.stream()
            .allMatch(s -> s.getStatoString().equals("DISPONIBILE")));
    }
}
```

Listing 5.2: Integration Test per StanzaDAO con reset dello stato

5.3 Testing Funzionale (Black-Box e Tracciabilità)

Il livello più alto di collaudo ha riguardato i `Controllers`. In questa fase (Black-Box Testing) la logica interna viene ignorata e il sistema viene testato esclusivamente in base agli input e agli output attesi.

Per garantire la massima **tracciabilità**, ogni test funzionale è stato mappato direttamente su uno specifico *Use Case Template* (Sezione 2.2). Sono stati collaudati sia i *Flussi Base* che i *Flussi Alternativi* (le eccezioni).

Il Listing 5.3 dimostra come il sistema gestisce fluidamente la *Race Condition* descritta nell'UC1 (Alternativa 3a), negando la prenotazione se lo slot risulta già occupato a database.

```java
class PrenotazioneControllerTest {
    private PrenotazioneController controller;
    private PrenotazioneDAOPostgres prenotazioneDAO;

    @BeforeEach
    void setUp() {
        DatabaseHelper.resetTestDatabase();
        prenotazioneDAO = new PrenotazioneDAOPostgres();
        StanzaDAOPostgres stanzaDAO = new StanzaDAOPostgres();
        controller = new PrenotazioneController(prenotazioneDAO, stanzaDAO);
    }

    @Test
    void testPrenotazioneNegataPerSlotOccupato_UC1_Alt3a() throws SQLException {
        // Setup: Il Cliente 2 tenta di prenotare uno slot appena occupato dal Cliente 1
```

```java
        String stanzaId = "R01";
        LocalDateTime slotRichiesto = LocalDateTime.of(2026, 10, 31,
            21, 0);

        // Simulo la race condition pre-occupando lo slot via DAO
        Prenotazione esistente = new Prenotazione(
            UUID.randomUUID().toString(), "CLIENTE_01", stanzaId,
            slotRichiesto, 3, 60.0, "CONFERMATA");
        prenotazioneDAO.save(esistente);

        // Execution & Assertion
        SlotNonDisponibileException eccezione = assertThrows(
            SlotNonDisponibileException.class,
            () -> controller.effettuaPrenotazione("CLIENTE_02",
                stanzaId, slotRichiesto, 4)
        );

        assertEquals("Lo slot orario selezionato non e' piu'
            disponibile.", eccezione.getMessage());
    }
}
```

Listing 5.3: Functional Test tracciato sull'UC1 (Flusso Alternativo 3a)

5.4 Conclusioni

L'approccio metodologico adottato, guidato dai principi dell'Object-Oriented Design e supportato dall'uso estensivo dei Design Pattern, ha permesso di sviluppare **EscapeManager** come un sistema robusto, altamente coeso e disaccoppiato. La suite di collaudo ha confermato la solidità della logica di business e la corretta integrazione con il layer relazionale, soddisfacendo integralmente i requisiti posti in fase di analisi e fornendo un'architettura facilmente estensibile per sviluppi futuri (es. integrazione di sensori fisici nelle stanze tramite *Adapter Pattern* o implementazione di un'interfaccia grafica avanzata al posto dell'attuale CLI).